UNIT IV – Knowledge & Reasoning

AI & ML for Mechanical Engineering (23A03701)

➤ Syllabus Map

- Statistical reasoning: probability, Bayes' theorem, certainty factors, Bayesian networks, Dempster-Shafer, fuzzy logic
- Knowledge representation: rule-based, procedural vs declarative, logic programming, forward & backward reasoning

1. Why Statistical Reasoning?

Real-world engineering data is ***uncertain, noisy and incomplete***. Sensors drift, a bearing may or may not be worn, a crack may or may not be present. Pure logic (TRUE/FALSE) cannot handle this, so AI uses reasoning under uncertainty.

➤ Sources of uncertainty

- Imprecise / noisy sensor readings (thermocouple error)
- Incomplete knowledge (not all symptoms observed)
- Vague concepts ("high temperature", "slightly worn")
- Unreliable expert rules ("usually", "often")

➤ Tools we study

Tool	Handles
Probability & Bayes	Randomness, belief update with evidence
Certainty Factors	Expert's confidence in a rule (MYCIN)
Bayesian Network	Many variables with dependencies
Dempster-Shafer	Ignorance (not knowing) vs disbelief
Fuzzy Logic	Vagueness / degrees of truth

2. Probability Basics

➤ Definition

Probability of event A: **$P(A) = (\text{favourable outcomes}) / (\text{total outcomes})$**, with $0 \leq P(A) \leq 1$, P(sure event) = 1, P(impossible) = 0.

➤ Important rules

- Complement: **$P(\neg A) = 1 - P(A)$**
- Addition: **$P(A \cup B) = P(A) + P(B) - P(A \cap B)$**
- Mutually exclusive: $P(A \cap B) = 0$
- Conditional: **$P(A \mid B) = P(A \cap B) / P(B)$**, $P(B) > 0$
- Product rule: $P(A \cap B) = P(A|B) \cdot P(B)$
- Independent events: $P(A \cap B) = P(A) \cdot P(B)$
- Total probability: $P(B) = \Sigma P(B|A_i) \cdot P(A_i)$

> ***Example.*** A lot has 100 shafts: 8 are defective. Probability that a randomly drawn shaft is good = 1 – 8/100 = 0.92. Two shafts drawn with replacement, both defective = 0.08 × 0.08 = 0.0064.

➤ Joint & marginal probability

Joint P(A,B) = both occur together. Marginal P(A) = sum of joint probabilities over all values of B. A full ***joint probability table*** answers any query but grows as 2^n for n Boolean variables – that is why Bayesian networks are needed.

3. Bayes' Theorem

➤ Bayes' Rule

$$P(H \mid E) = P(E \mid H) \cdot P(H) / P(E)$$

- P(H) = ***prior*** – belief in hypothesis before evidence
- P(E|H) = ***likelihood*** – chance of evidence if H is true
- P(H|E) = ***posterior*** – updated belief after evidence
- P(E) = P(E|H)P(H) + P(E|¬H)P(¬H) (normaliser)

Derivation: $P(H \cap E) = P(H|E)P(E) = P(E|H)P(H)$. Equate and divide by $P(E)$.

➤ Worked Example 1 – Crack detection

1% of turbine blades have a crack. An ultrasonic test detects a crack with 95% probability if present (sensitivity), and gives a false alarm 4% of the time on healthy blades. A blade tests ***positive***. P(crack | positive)?

$P(H)=0.01$, $P(E|H)=0.95$, $P(E|\neg H)=0.04$

$P(E) = 0.95\times0.01 + 0.04\times0.99 = 0.0095 + 0.0396 = 0.0491$

$P(H|E) = 0.0095 / 0.0491 = 0.1935 \approx 19.4\ \%$

Even a positive test gives only ~19% – because cracks are rare (low prior).

➤ Worked Example 2 – Machine source

Machine M1 makes 60% of parts (3% defective), M2 makes 40% (5% defective). A defective part is found. Probability it came from M1?

$P(D) = 0.6\times0.03 + 0.4\times0.05 = 0.018 + 0.020 = 0.038$

$P(M1|D) = 0.018/0.038 = 0.4737$; so $P(M2|D) = 0.5263$.

➤ Limitations of pure Bayes

- Needs many prior and conditional probabilities (hard to obtain)
- Assumes independence of evidences (naive Bayes) otherwise table explodes
- Cannot represent ignorance – all probability must be assigned

4. Certainty Factors & Rule-Based Systems

Developed for the medical expert system ***MYCIN*** (diagnosing blood infections). Experts give confidence numbers instead of exact probabilities.

➤ Certainty Factor (CF)

$$CF[h, e] = MB[h, e] - MD[h, e]$$

- MB = measure of belief (0 to 1) – evidence supports h
- MD = measure of disbelief (0 to 1) – evidence opposes h
- CF ranges from **–1** (absolutely false) through **0** (no information) to **+1** (absolutely true)

➤ Rule form

IF engine_vibration is high AND bearing_temp is high
THEN bearing_failure CF = 0.7

➤ Combining rules

- ***Conjunction*** (AND): CF(premise) = $\min(CF_1, CF_2)$
- ***Disjunction*** (OR): CF(premise) = $\max(CF_1, CF_2)$
- ***Rule propagation:*** CF(conclusion) = CF(rule) × CF(premise)
- ***Two rules, same conclusion*** (CF_1, CF_2):
  - both positive: $CF = CF_1 + CF_2(1 - CF_1)$
  - both negative: $CF = CF_1 + CF_2(1 + CF_1)$
  - opposite signs: $CF = (CF_1 + CF_2) / (1 - \min(|CF_1|,|CF_2|))$

➤ Example

Rule 1: IF vibration high (CF 0.8) AND temp high (CF 0.6) THEN failure, rule CF = 0.7.

Premise = min(0.8, 0.6) = 0.6 → CF_1 = 0.7 × 0.6 = 0.42

Rule 2: IF oil_debris present (CF 0.9) THEN failure, rule CF = 0.5 → CF_2 = 0.45

Combined = 0.42 + 0.45(1 − 0.42) = 0.42 + 0.261 = 0.681

➤ Rule-based system architecture

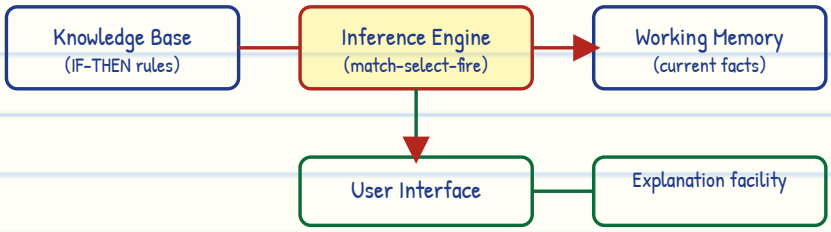


Fig 1: Components of a rule-based (expert) system

➤ Advantages / disadvantages

- \+ Easy to understand, modular, explains its reasoning (why / how)
- \+ Handles uncertainty approximately
- − CF are ad-hoc (not rigorous probability); rule interactions can mislead
- − Large rule bases hard to maintain

5. Bayesian Networks (Belief Networks)

➤ Definition

A ***Bayesian network*** is a directed acyclic graph (DAG) in which nodes are random variables and an arrow $X \rightarrow Y$ means X directly influences Y. Each node has a ***conditional probability table (CPT)*** $P(X_i \mid \text{Parents}(X_i))$.

➤ Joint distribution

$$P(X_1,..,X_n) = \Pi\ P(X_i \mid \text{Parents}(X_i))$$

Needs far fewer numbers than the full 2^n joint table.

➤ Example: machine overheating

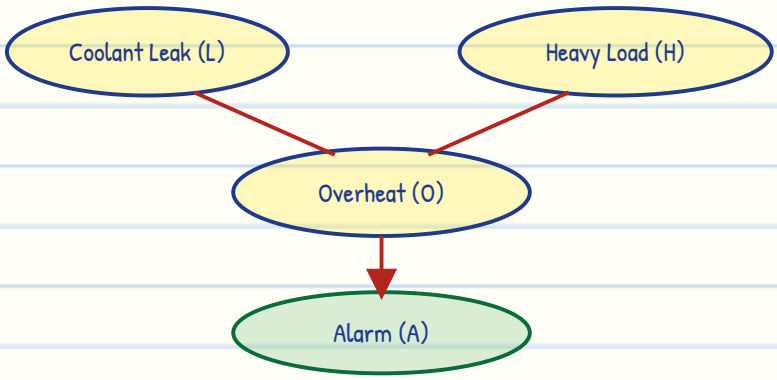


Fig 2: Bayesian network – L and H cause O; O triggers A

Node	CPT values
P(L)	0.02
P(H)	0.30
P(O | L,H)	L,H: 0.95 | L,¬H: 0.80 | ¬L,H: 0.30 | ¬L,¬H: 0.01
P(A | O)	O: 0.90 | ¬O: 0.05

➤ Compute P(L, H, O, A)

P(L=T, H=T, O=T, A=T) = P(L)·P(H)·P(O|L,H)·P(A|O)

= 0.02 × 0.30 × 0.95 × 0.90 = 0.00513

➤ Compute P(O) (marginal)

P(L,H)=0.006, P(L,¬H)=0.014, P(¬L,H)=0.294, P(¬L,¬H)=0.686

P(O) = 0.006(0.95)+0.014(0.80)+0.294(0.30)+0.686(0.01)

= 0.0057+0.0112+0.0882+0.00686 = 0.11196

P(A) = 0.90(0.11196) + 0.05(0.88804) = 0.10076 + 0.04440 = **0.1452**

➤ Inference types

- ***Causal*** (top-down): cause → effect, e.g. P(A | L)
- ***Diagnostic*** (bottom-up): effect → cause, e.g. P(L | A) using Bayes
- ***Explaining away:*** if overheating is explained by heavy load, belief in leak drops

Methods: exact (enumeration, variable elimination), approximate (sampling). Applications: fault diagnosis, reliability, condition monitoring, medical diagnosis.

6. Dempster-Shafer Theory (DST)

A generalisation of probability that allows ***"I don't know"***. Evidence is assigned not to single outcomes but to ***subsets of hypotheses***. Also called the theory of belief functions / evidence theory.

➤ Key terms

- ***Frame of discernment*** **Θ**: set of all mutually exclusive possible hypotheses, e.g. Θ = {Bearing, Gear, Shaft}
- ***Power set 2^Θ***: all subsets of Θ (2^3 = 8 here)
- ***Basic probability assignment (mass) m***: m: $2^{\Theta} \rightarrow [0,1]$, $m(\varnothing)=0$, $\Sigma\, m(A) = 1$
- ***Belief Bel(A)*** = $\Sigma\, m(B)$ for all $B \subseteq A$ (total support, lower bound)
- ***Plausibility Pl(A)*** = $\Sigma\, m(B)$ for all $B \cap A \neq \varnothing$ = 1 – Bel(¬A) (upper bound)
- ***Belief interval*** [Bel(A), Pl(A)]; width = ignorance

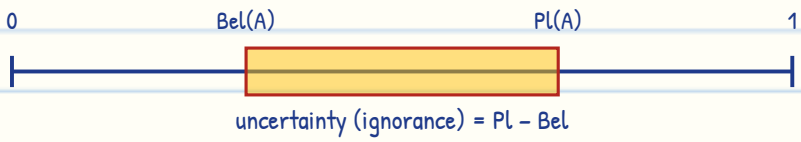


Fig 3: Belief interval

➤ Example – single source

Sensor 1 says fault is in Bearing or Gear: $m_1(\{B,G\}) = 0.6$, $m_1(\Theta) = 0.4$ (the rest is ignorance). Bel({B,G}) = 0.6; Pl({B,G}) = 0.6 + 0.4 = 1.0. Bel({B}) = 0; Pl({B}) = 1.0.

➤ Dempster's rule of combination

$$m_{12}(C) = \Sigma_{A \cap B = C}\ m_1(A) \cdot m_2(B) \ / \ (1 - K)$$

where conflict $K = \Sigma_{A \cap B = \varnothing}\ m_1(A) \cdot m_2(B)$. Dividing by (1–K) re-normalises.

➤ Worked example

Θ = {B, G, S}. Sensor 1: $m_1(\{B,G\}) = 0.6$, $m_1(\Theta) = 0.4$. Sensor 2: $m_2(\{G,S\}) = 0.7$, $m_2(\Theta) = 0.3$.

	$m_2\{G,S\}=0.7$	$m_2\Theta=0.3$
$m_1\{B,G\}=0.6$	{G} : 0.42	{B,G} : 0.18
$m_1\Theta=0.4$	{G,S} : 0.28	Θ : 0.12

No empty intersections → K = 0, so no normalising.

Result: $m(\{G\})=0.42$, $m(\{B,G\})=0.18$, $m(\{G,S\})=0.28$, $m(\Theta)=0.12$

Bel({G}) = 0.42, Pl({G}) = 1.0 (all sets contain G). Bel({B,G}) = 0.42+0.18 = 0.60; Pl({B,G}) = 1.0.

➤ DST vs Bayesian probability

Bayesian	Dempster-Shafer
Probability on single outcomes	Mass on subsets of outcomes
P(A)+P(¬A)=1 always	Bel(A)+Bel(¬A) ≤ 1
Ignorance spread equally (prior)	Ignorance kept explicit in m(Θ)
Needs priors	No priors needed
Single number	Interval [Bel, Pl]

Use: sensor fusion, multi-sensor fault diagnosis, decision under ignorance.

7. Fuzzy Logic

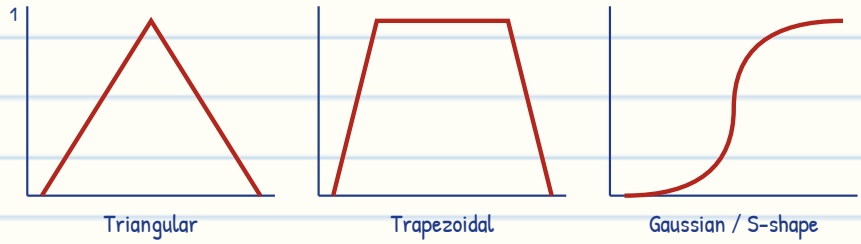
Proposed by **Lotfi Zadeh (1965)**. Classical (crisp) logic says an element is either in a set or not. Fuzzy logic allows degrees of membership between 0 and 1, modelling human-like vague terms: "warm", "fast", "high pressure".

➤ Fuzzy set

A fuzzy set A in universe X is a set of pairs: $A = \{ (x, \mu_A(x)) \mid x \in X \}$, where $\mu_A(x) \in [0,1]$ is the membership function.

Crisp set: μ is only 0 or 1. Example: "Hot" = {(30°C,0.2), (40°C,0.6), (50°C,1.0)}.

➤ Common membership functions



Triangular: μ(x)=0 for x≤a, (x−a)/(b−a) for a≤x≤b, (c−x)/(c−b) for b≤x≤c, 0 for x≥c.

➤ Fuzzy set operations (Zadeh)

Operation	Formula
Union (OR)	μ_{A∪B}(x) = max[μ_A(x), μ_B(x)]
Intersection (AND)	μ_{A∩B}(x) = min[μ_A(x), μ_B(x)]
Complement (NOT)	μ_{A'}(x) = 1 − μ_A(x)
Equality / subset	A=B if μ_A=μ_B ∀x; A⊆B if μ_A ≤ μ_B

Example. A = {0.2/a, 0.7/b, 1/c}, B = {0.5/a, 0.4/b, 0.9/c}

A∪B = {0.5/a, 0.7/b, 1/c}; A∩B = {0.2/a, 0.4/b, 0.9/c}; $A^{'}$ = {0.8/a, 0.3/b, 0/c}

➤ Crisp vs fuzzy

- Crisp: temperature 40°C is "hot" TRUE/FALSE with a sharp boundary
- Fuzzy: 40°C is hot to 0.6 degree and warm to 0.4 degree – smooth transition

8. Fuzzy Inference System (FIS)

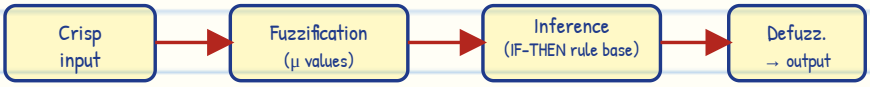


Fig 4: Fuzzy logic controller (Mamdani)

➤ Steps

- **1. Fuzzification:** convert crisp inputs to membership degrees of linguistic variables (Cold, Warm, Hot)

- **2. Rule evaluation:** apply IF-THEN rules; AND = min, OR = max
- **3. Aggregation:** combine the clipped output sets of all rules (max)
- **4. Defuzzification:** convert the fuzzy output to one crisp value

➤ Defuzzification methods

- **Centroid (centre of gravity):** $z^* = \int \mu(z) \cdot z \, dz / \int \mu(z) \, dz$ - most used
- Weighted average (discrete): $z^* = \Sigma \mu_i \cdot z_i / \Sigma \mu_i$
- Mean of maxima, smallest / largest of maxima, bisector

➤ Example – Fan speed controller for a motor

Input: temperature T. Sets: Cold, Warm, Hot. Output: fan speed Slow (20%), Medium (50%), Fast (90%).

R1: IF T is Cold THEN speed is Slow
R2: IF T is Warm THEN speed is Medium
R3: IF T is Hot THEN speed is Fast

At T = 65°C fuzzification gives: Cold = 0, Warm = 0.7, Hot = 0.3.

Weighted average: speed = (0.7×50 + 0.3×90) / (0.7+0.3) = (35+27)/1 = 62 %

➤ Rule with two inputs

IF temperature is High AND pressure is Medium THEN valve opening is Large. If μ_High(T)=0.8, μ_Med(P)=0.5 → firing strength = min(0.8,0.5) = 0.5, so the Large set is clipped at 0.5.

➤ Applications in Mechanical Engg.

- Washing machine, air conditioner, refrigerator controllers
- Automatic gearbox / ABS brakes, cruise control
- Robot path planning, CNC process parameter selection
- Furnace temperature control, fault diagnosis

➤ Pros & cons

- + No exact mathematical model needed; easy to tune by expert knowledge
- + Robust to noisy inputs
- – Rule base grows with inputs; membership design is subjective; no learning ability (hence neuro-fuzzy)

9. AI for Knowledge Representation (KR)

➤ Knowledge Representation

KR is the way of ***encoding knowledge about the world in a form a computer can store and reason with*** to solve complex tasks.

➤ Kinds of knowledge

- ***Objects*** (facts): a bolt is a fastener
- ***Events***: pump failed at 10:00
- ***Performance***: how to do things
- ***Meta-knowledge***: knowledge about knowledge

➤ Requirements of a good KR scheme

- ***Representational adequacy*** – can represent all needed knowledge
- ***Inferential adequacy*** – can derive new knowledge
- ***Inferential efficiency*** – can direct inference to promising paths
- ***Acquisitional efficiency*** – easy to add new knowledge

➤ Techniques of KR

Technique	Idea	Example
Logical (propositional / predicate)	Facts as logical sentences	∀x Motor(x) → HasRotor(x)
Semantic network	Graph of nodes and labelled links	Car –is-a→ Vehicle
Frames	Slot-filler structures with inheritance	Frame Pump: type, flow, head
Production rules	IF-THEN rules	IF temp>90 THEN alarm

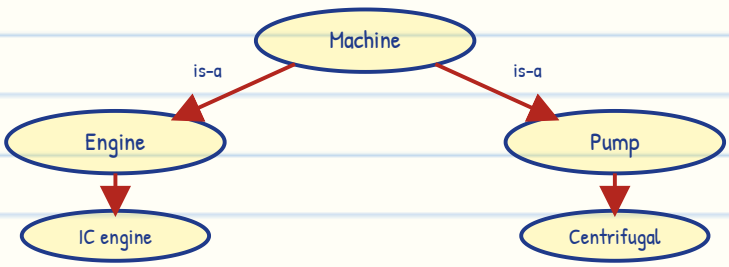


Fig 5: Semantic network (inheritance)

10. Rule-Based Knowledge Representation

Knowledge is stored as ***production rules: IF (condition) THEN (action/conclusion)***. The condition part is the *antecedent* (LHS) and the conclusion is the *consequent* (RHS).

```
R1: IF  noise = abnormal AND temp = high  THEN bearing = faulty
R2: IF  bearing = faulty                  THEN action = replace_bearing
R3: IF  oil_level = low                   THEN action = add_oil
```

➤ Components

- ***Rule base*** (long-term memory), ***working memory*** (facts about the current case), ***inference engine*** (cycle: match → conflict resolution → act)

➤ Conflict-resolution strategies

- Specificity (more specific rule first), recency, priority, first-match

➤ Pros and cons

- \+ Modular, natural, easy to update, transparent
- – Slow with many rules, hard to see rule interactions, poor at learning

11. Procedural vs Declarative Knowledge

Procedural ("know-how")	Declarative ("know-what")
Describes **how** to do something	Describes **what** is true (facts)
Control information embedded in knowledge	Control separate from knowledge
Written as procedures / programs / code	Written as facts, rules, logic statements
Efficient to run, hard to modify	Easy to modify, less efficient
Hard to reuse for other purposes	Same fact can answer many queries
Eg: algorithm to start an engine	Eg: "Engine has 4 cylinders"
Languages: C, Java, Python	Languages: Prolog, rule engines

Real systems mix both: facts are declarative, inference/search strategy is procedural.

12. Logic Programming

A programming style where the program is a ***set of logical statements (facts and rules)*** and computation is logical inference (theorem proving). The programmer states *what* is true; the system decides *how* to answer queries. Best known language: ***Prolog*** (PROgramming in LOGic), based on Horn clauses and resolution with unification and backtracking.

➤ Prolog building blocks

- ***Fact:*** unconditional truth - `motor(m1).`
- ***Rule:*** head :- body. (head is true if body is true) - `:-` means "if", comma means AND, semicolon means OR
- ***Query:*** a question - `?- faulty(m1).`
- Variables begin with a capital letter (X, Machine); constants with lower case
- Every clause ends with a full stop

➤ Example program

```
% facts
vibration_high(m1).
temp_high(m1).
temp_high(m2).
oil_low(m3).

% rules
bearing_faulty(X) :- vibration_high(X), temp_high(X).
needs_service(X)  :- bearing_faulty(X).
needs_service(X)  :- oil_low(X).
```

Query `?- needs_service(m1).` → ***true*** (via bearing_faulty).

Query `?- needs_service(Z).` → Z = m1 ; Z = m3. (backtracking gives all answers)

Query `?- bearing_faulty(m2).` → ***false*** (no vibration fact; closed-world assumption)

➤ Key concepts

- ***Unification:*** matching terms by binding variables, e.g. needs_service(X) with needs_service(m1) gives X = m1

- **Resolution:** inference rule used to derive new clauses / prove goals
- **Backtracking:** if a path fails, return to the last choice point and try the next clause
- **Horn clause:** clause with at most one positive literal – $A \leftarrow B_1 \wedge \ldots \wedge B_n$
- **Closed-world assumption:** anything not provable is taken as false (negation as failure)

➤ Recursion example

```
parent(a,b).  parent(b,c).
ancestor(X,Y) :- parent(X,Y).
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Applications: expert systems, natural-language processing, planning, theorem proving, database queries.

13. Forward and Backward Reasoning

➤ Forward chaining

Data-driven: start from known facts, fire rules whose conditions match, add conclusions, repeat until goal reached or nothing new.

➤ Backward chaining

Goal-driven: start from the goal (hypothesis), find rules that conclude it, make their conditions new sub-goals, until sub-goals match facts.

➤ Rule base

```
R1: IF A AND B THEN C
R2: IF C THEN D
R3: IF D AND E THEN F
Facts: A, B, E.   Goal: F
```

Forward: facts {A,B,E} → R1 fires, adds C → R2 fires, adds D → R3 fires (D,E known), adds F. Goal reached.

Backward: Goal F ← R3 needs D and E. E is a fact ✔. D ← R2 needs C. C ← R1 needs A, B – both facts ✔. So F proved.

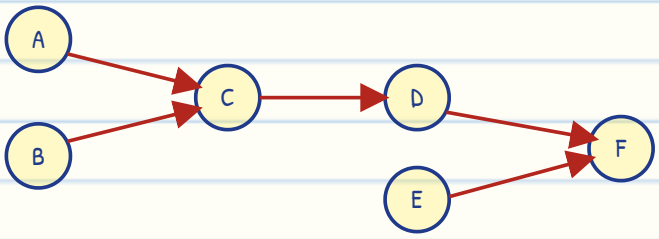


Fig 6: Inference graph for the rule base

➤ Comparison

Forward chaining	Backward chaining
Data-driven, bottom-up	Goal-driven, top-down
Starts from facts → conclusions	Starts from goal → supporting facts
Good for monitoring, planning, design, control	Good for diagnosis, proving a specific hypothesis
May derive many irrelevant facts	Explores only relevant rules
Eg: OPS5, CLIPS, process control	Eg: Prolog, MYCIN
Breadth-first nature	Depth-first nature

Bidirectional / mixed: combine both – work from facts and goal until they meet.

14. Quick Revision Formulas

- Bayes: $P(H|E) = P(E|H)P(H)/P(E)$
- CF: CF = MB – MD; premise AND = min, OR = max; conclusion = CF_rule × CF_premise
- Combine CFs (both +ve): $CF_1 + CF_2(1 - CF_1)$
- BN: $P(x_1..x_n) = \Pi P(x_i|\text{parents})$
- DST: $Bel(A)=\Sigma_{B\subseteq A}m(B)$; $Pl(A)=\Sigma_{B\cap A\neq\emptyset}m(B)$; K = conflict mass
- Fuzzy: U = max, ∩ = min, complement = 1 – μ; centroid $z^* = \Sigma\mu z/\Sigma\mu$

15. Important Exam Questions (Short Q & A)

Q1. State Bayes' theorem and explain each term.

$P(H|E)=P(E|H)P(H)/P(E)$: posterior = likelihood × prior / evidence probability. It updates belief in H after observing E.

Q2. What is a certainty factor? What is its range?

A number expressing the confidence in a hypothesis given evidence, CF = MB − MD, ranging from −1 to +1.

Q3. What is a Bayesian network? Why is it useful?

A DAG of variables with CPTs; it gives a compact representation of the joint distribution using conditional independence, avoiding a 2^n table.

Q4. Differentiate belief and plausibility in Dempster-Shafer theory.

Belief = total evidence that directly supports A (lower bound). Plausibility = evidence that does not contradict A (upper bound). Pl − Bel = ignorance.

Q5. What is the difference between fuzzy and crisp sets?

Crisp: membership is 0 or 1 with sharp boundary. Fuzzy: membership is any value in [0,1], giving gradual transition.

Q6. List the stages of a fuzzy inference system.

Fuzzification → rule evaluation (inference) → aggregation → defuzzification.

Q7. Distinguish procedural and declarative knowledge.

Procedural = how-to, embedded control (code). Declarative = facts, separate from control (Prolog, rules).

Q8. Compare forward and backward chaining.

Forward is data-driven, bottom-up; backward is goal-driven, top-down. Diagnosis uses backward; monitoring uses forward.

Q9. What is Prolog? Write a fact and a rule.

A logic programming language. Fact: `motor(m1).` Rule: `hot(X) :- motor(X), temp_high(X).`

Q10. Write about the characteristics of a good knowledge representation scheme.

Representational adequacy, inferential adequacy, inferential efficiency and acquisitional efficiency.

Long questions to prepare:

- Explain Bayes' theorem with a numerical example and its use in fault diagnosis.
- Explain Dempster-Shafer theory and Dempster's rule of combination with an example.
- Explain the architecture of a fuzzy logic controller with a mechanical example.
- Explain forward and backward reasoning with an example; compare them.
- Describe rule-based systems and certainty-factor propagation.

~ End of Unit IV ~ All the best! ~